Econometrics Softwares

Session 3: Communicating Results in R

Mathilde Col & Jérémy Do Nascimento Miguel

University of Bordeaux

This session

- Regression analysis using *fixest*
  - Estimating models
  - Exporting clean tables
- Visualization in R
  - Syntax of **ggplot2**
  - Doing clean graphs

Data

- Subsample of the 2015 Ethiopian AgSS in Oromia
- "AGSS_2015" file in dta format
  - **zone**: agroecological zone id
  - **hhsex**: household head sex. 0 for Female; 1 for Male
  - **woreda_id**: woreda id
  - **hhid**: household id
  - **AREAH**: farm size in hectare
  - **PRODQ**: production in quintals
  - **OWNTYPE1**: plot ownership status. 0 if rented; 1 for ownership
  - **EXT1**: access to extension program. 0 if not; 1 if yes.
  - **IRRG**: usage of irrigation. 0 for no; 1 for yes
  - **FERT**: usage for fertilizer. 0 for no; 1 for yes
  - **D21B**: quantity of fertilizer used in kg.

Regressions

The *fixest* package

You can download (and load) the **fixest** package from the CRAN, and find a quick introduction here. It provides in one single package a family of functions to perform estimations with multiple fixed-effects.

The main functions are:

- **feols** for linear models
- **feglm** for generalized linear models (e.g. logit) $\rightarrow$ you specify which model with the option **family** $=$
- **femlm** performs direct maximum likelihood estimation
- **feNmlm** extends the latter to allow the inclusion of non-linear in parameters right-hand-sides
- **fepois** and **fenegbin** are aliases for Poisson and negative binomial fixed-effect estimations.

Linear regression

Let's take the dataframe from Slide 3 as an example. We want to know whether access to the extension program increased fertilizer use and the production in quintals.

```
df <- read_xlsx("data/_raw/AGSS_2015.xlsx")
reg_1var <- feols(FERT ~ EXT1, data = df)
reg_control <- feols(FERT ~ EXT1 + hhsex, data = df)
reg_2var <- feols(c(FERT, PRODQ) ~ EXT1, data = df)
# If you want to add more than 1 outcome, you can just list them
# with c(var1, var2, var3)
```

Now, we want to add a zone fixed effect:

```
reg_fe <- feols(c(FERT, PRODQ) ~ EXT1 | zone, data = df)
```

Be careful, it automatically assumes that you cluster your standard errors at the fixed effect level.

Regression output

To have the result of a regression, you can use **etable(name of the regression object)**:

```
etable(reg_fe)
                                  reg_fe.1          reg_fe.2
Dependent Var.:                       FERT             PRODQ

EXT1                   0.5483*** (0.0337) 0.8779* (0.3874)
Fixed-Effects:         ------------------ ----------------
zone                                  Yes               Yes
_______________ __________________ ________________
S.E.: Clustered                  by: zone          by: zone
Observations                       10,125            10,125
R2                                0.11109           0.01428
Within R2                         0.03621           0.00069
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Regression output

You can combine more than one regression in the output:

```
etable(reg_fe, reg_1var)
                                  reg_fe.1          reg_fe.2           reg_1var
Dependent Var.:                       FERT             PRODQ               FERT

EXT1                   0.5483*** (0.0337) 0.8779* (0.3874) 0.5755*** (0.0289)
Constant                                                    0.3222*** (0.0047)
Fixed-Effects:         ------------------ ---------------- ------------------
zone                                  Yes               Yes                 No
_______________ __________________ ________________ __________________
S.E. type                        by: zone          by: zone                IID
Observations                       10,125            10,125             10,125
R2                                0.11109           0.01428            0.03764
Within R2                         0.03621           0.00069                 --
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Linear regression

Finally, we want to cluster at the woreda level:

```
reg2 <- feols(c(FERT, PRODQ) ~ EXT1 | zone,
    cluster = ~ woreda_id,
    data = df)
```

Other features:

- If you have more than one fixed effect, you can just add them with a + sign:

```
reg_2fe <- feols(FERT ~ EXT1 | FE1 + FE2, data = df)
```

- You can also split the samples by adding the option **split** = ~ **variable**.

```
reg_split <- feols(FERT ~ EXT1, split = ~ hhsex, data = df)
```

Generalized linear models

If you use **feglm()**, you have an additional option to add, **family** =

- binomial(link = "logit")
- gaussian(link = "identity")
- Gamma(link = "inverse")
- inverse.gaussian(link = "1/mu")
- poisson(link = "log")
- quasi(link = "identity", variance = "constant")
- quasibinomial(link = "logit")
- quasipoisson(link = "log")

By default, the **family** option is set on poisson.

```
logit <- feglm(binary ~ x,
    family = binomial(link = "logit"),
    data = df)
```

Instrumental variables

The **fixest** package provides also a very useful function for instrumental variables.

Imagine the extension program $EXT1$ is endogenous, and you have an instrument called *instrument*, exogeneous to the outcomes, that explains $EXT1$:

```
reg_iv <- feols(c(FERT, D21B) ~ t | EXT1 ~ instrument,
    data = df)
```

The output will the second-stage estimation.

Export table

You just have to add the option **tex = TRUE** in the **etable()**:

```
> etable(reg2, tex = T)
\begingroup
\centering
\begin{tabular}{lcc}
   \tabularnewline \midrule \midrule
   Dependent Variables: & FERT           & PRODQ\\
   Model:               & (1)            & (2)\\
   \midrule
   \emph{Variables}\\
   EXT1                 & 0.5483$^{***}$ & 0.8779$^{***}$\\
                        & (0.0289)       & (0.3094)\\
   \midrule
   \emph{Fixed-effects}\\
   zone                 & Yes            & Yes\\
   \midrule
   \emph{Fit statistics}\\
   Observations         & 10,125         & 10,125\\
   R$^2$                & 0.11109        & 0.01428\\
   Within R$^2$         & 0.03621        & 0.00069\\
   \midrule \midrule
   \multicolumn{3}{l}{\emph{Clustered (woreda\_id) standard-errors in parentheses}}
\\
   \multicolumn{3}{l}{\emph{Signif. Codes: ***: 0.01, **: 0.05, *: 0.1}}\\
\end{tabular}
\par\endgroup
```

Export table

If you specify a file destination, it will store a .tex file directly: etable(reg2, tex = T, file = "output/table/export1.tex")

You can customize almost everything (more details here), to get a result that looks like this:

Language tested	Local language		French	
Dependent Variables:	Writing	Reading	Writing	Reading
Model:	(1)	(2)	(3)	(4)
Variables				
Born after 1994	0.008	0.007	0.138***	0.137***
	(0.012)	(0.011)	(0.020)	(0.020)
High exposed commune	0.023	0.022	0.034	0.032
	(0.015)	(0.015)	(0.037)	(0.037)
Born after 1994 × High exposed commune	0.039**	0.039**	0.055*	0.054*
	(0.018)	(0.017)	(0.028)	(0.028)
Fixed-effects				
District FE	Yes	Yes	Yes	Yes
Fit statistics				
Mean of Y	0.123	0.118	0.489	0.484
Observations	8,636	8,623	8,636	8,633
R^2	0.04228	0.04123	0.15238	0.14913

Clustered (LSMS cluster) standard-errors in parentheses

*Signif. Codes: ***: 0.01, **: 0.05, *: 0.1*

Notes: All dependent variables are binary outcomes. The model estimated is a linear regression. The level of analysis is at the individual level, i.e. one observation is one person. The differences in the number of observations is related to missingness of the outcome of interest in the raw LSMS data. I use the weights given in the LSMS data, and cluster the standard errors at the enumeration area. Clustering at the treatment level, i.e. the commune, produces the same results.

Table 2: TWFE results on learning outcomes

Exercise 1

Use the same dataframe AGSS_2015:

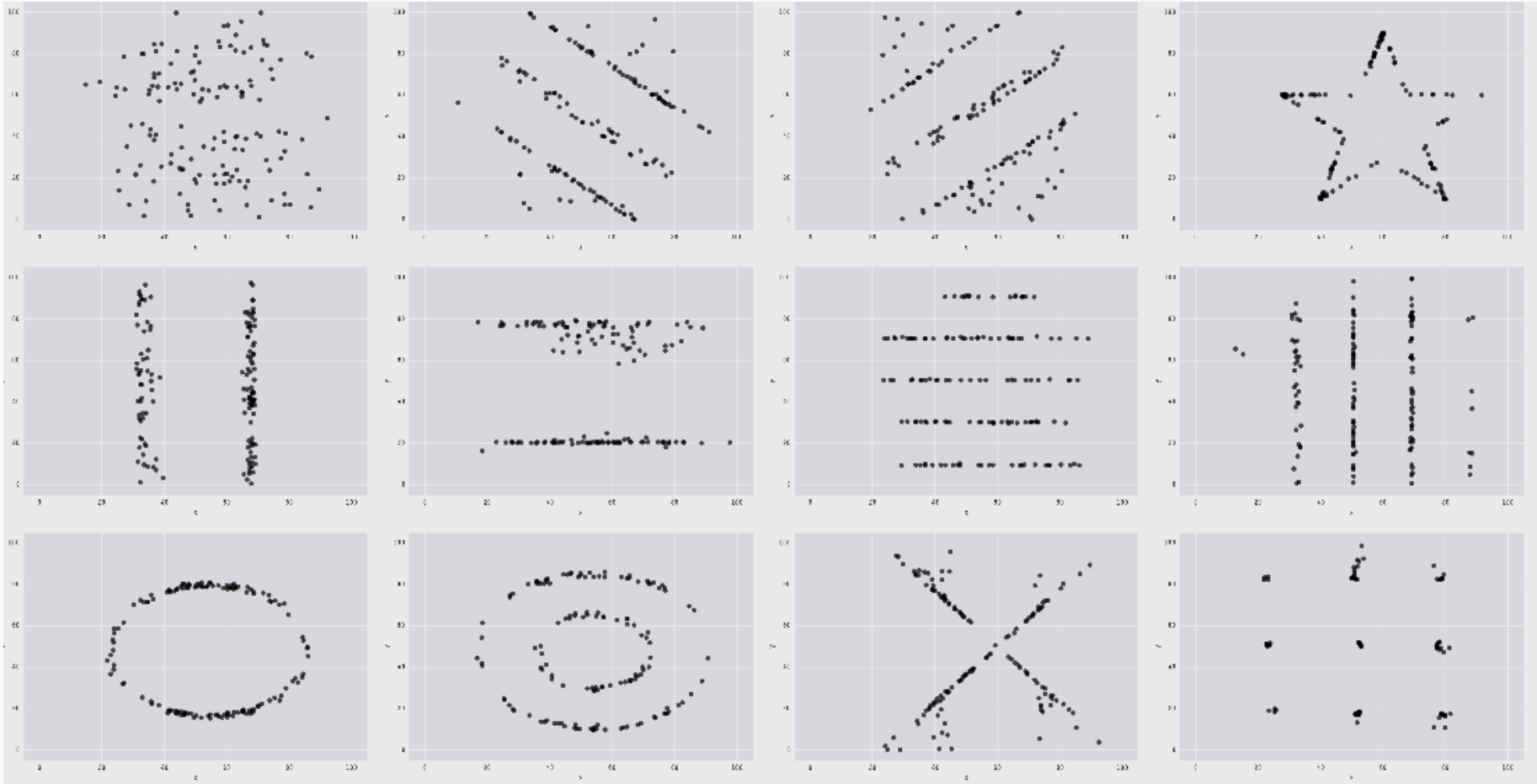
- Missing values for the quantity of fertilizer (D21B) are coded as 9999.999, recode them such that it does not alter your analysis

We want to do the following analysis:

- Using a linear regression, estimate whether the access to the extension program (EXT1) is correlated with the quantity of fertilizer (D21B).
  - Add a control for the farm size (AREAH)
  - Add woreda fixed effects (woreda_id)
  - Cluster your standard errors at the zone level (zone)
  - Export your results
- Using a logit model, estimate whether the sex of the household head (hhsex) is correlated with access to the extension program (EXT1)
  - Follow the same three steps
  - Export the table in a .tex file

Visualization

Visualizations in R

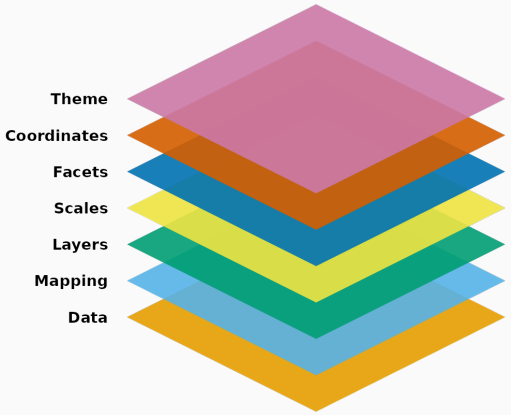


- Source : Data Analysis and Visualization in R

Grammar of graphics

The main principle is that graphics are composed of different layers[1]

1. Data: the data we want to plot
2. Aesthetics: scales onto which the data is mapped
3. Geometries: shapes used to represent the data
4. Facets: Rows and columns of sub-plots
5. Statistics: Statistical models and summaries
6. Coordinates: Plotting space for the data
7. Themes: All the formatting not related to the data



[1] Reference: Hadley Wickham, et al., *ggplot2: Elegant Graphics for Data Analysis*, online

ggplot2 Overview: ggplot()

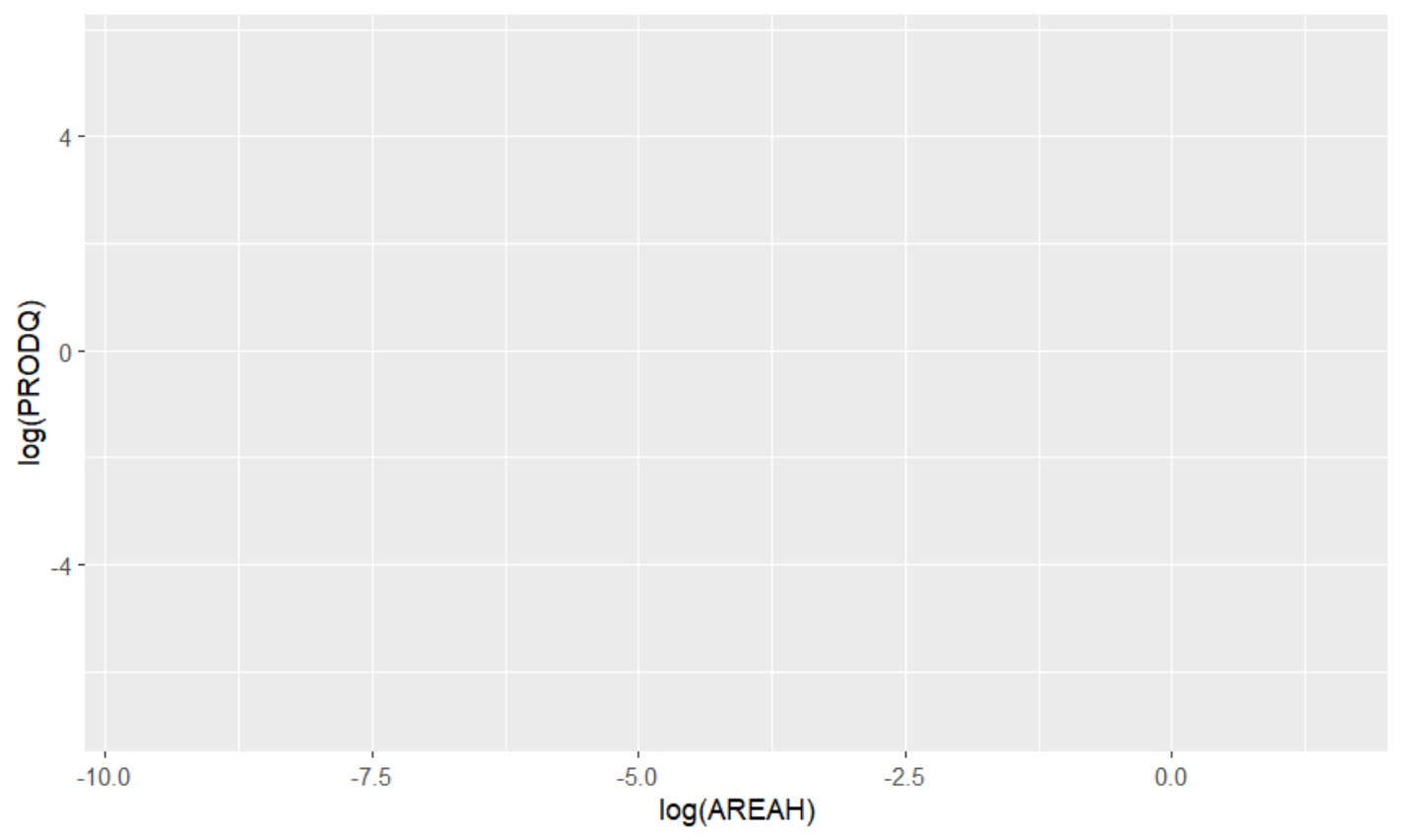
We can start a plot with the **ggplot()** function from the **ggplot2** package

```
library(ggplot2)
data <- read.csv("data/AGSS_15.csv", sep = ",")
ggplot(data)
```

ggplot2 Overview: aesthetics

We add the dimensions to the graph

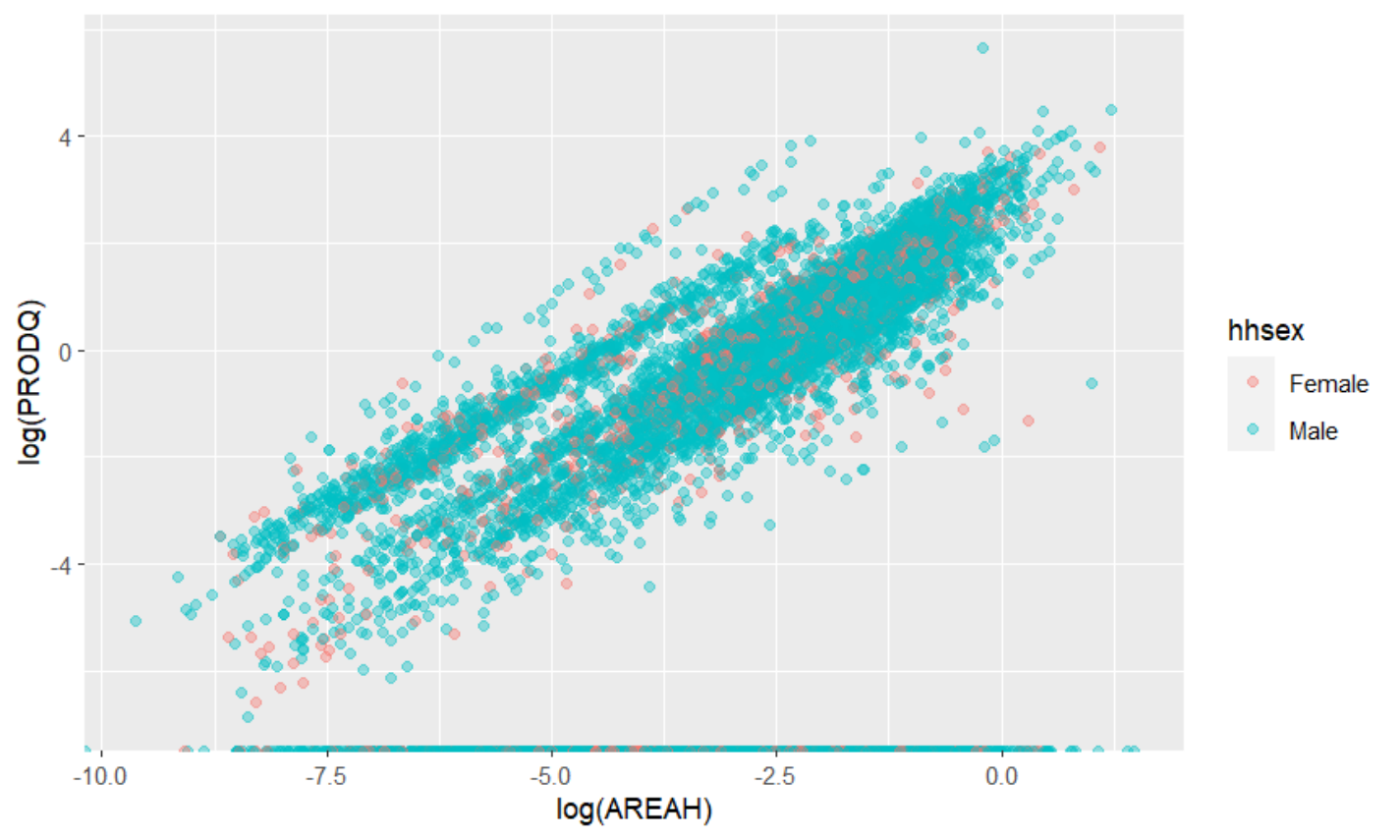
```
plot1 <- ggplot(data) +
  aes(x = log(AREAH), y = log(PRODQ), colour = hhsex)
```



ggplot2 Overview: geometries

We add the layers to represent these dimensions

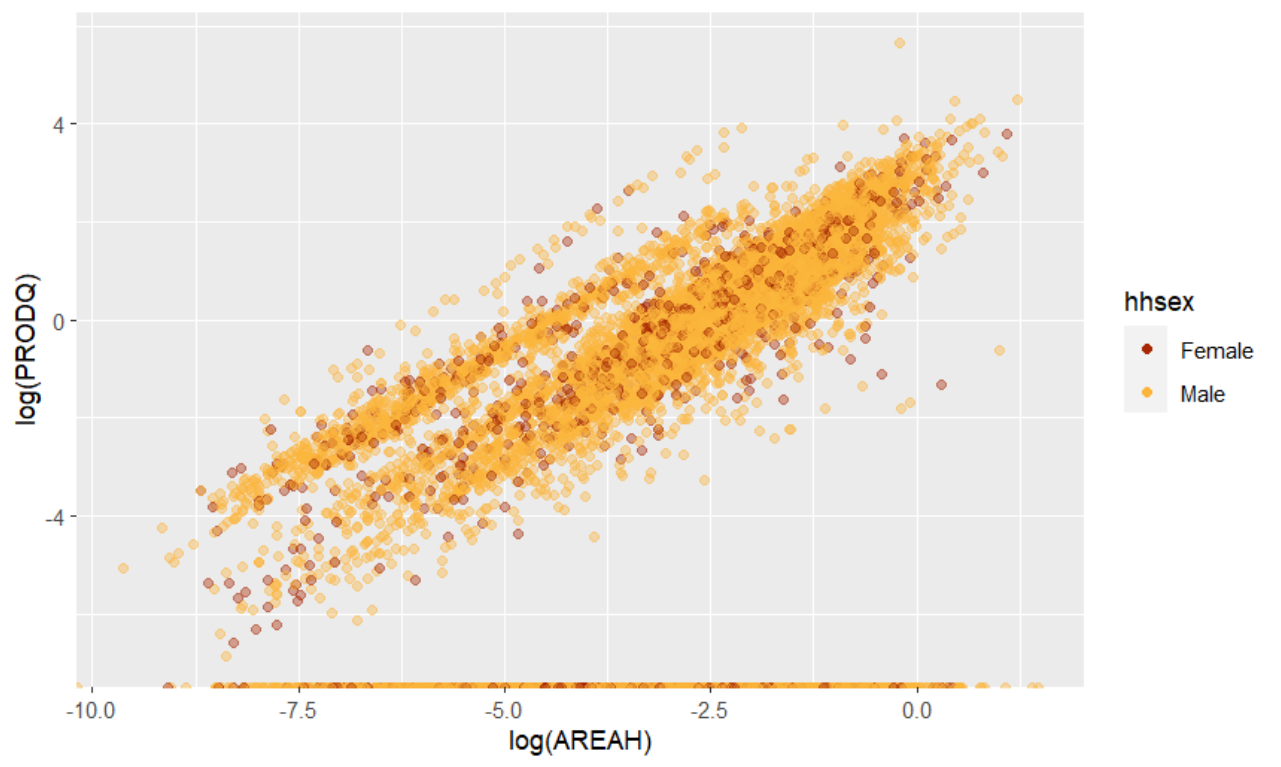
```
plot2 <- plot1 +
  aes(x = log(AREAH), y = log(PRODQ), colour = hhsex) +
  geom_point(alpha = .4)
  # alpha = transparency
```



ggplot2 Overview: scales

We change the scale colours (and names):

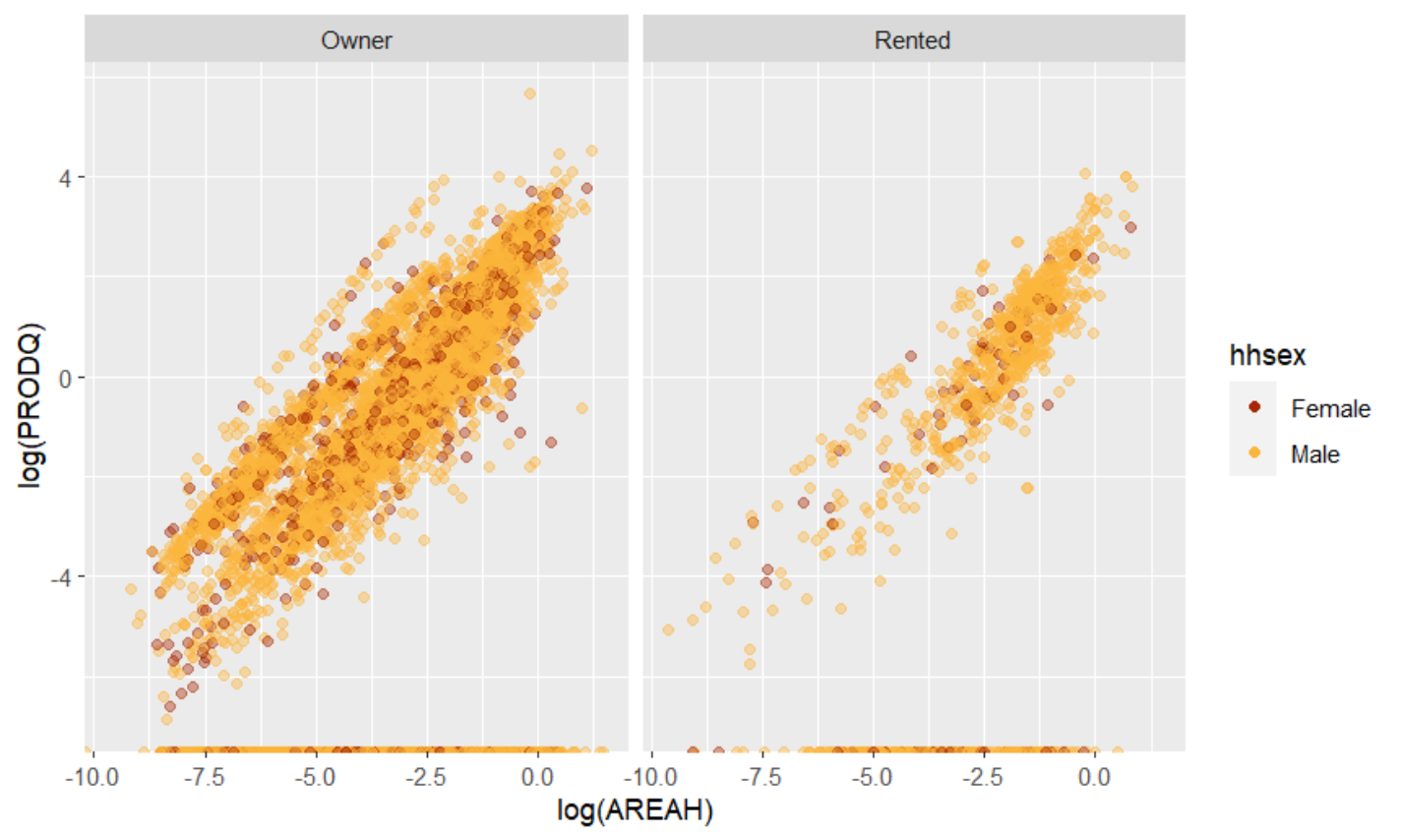
```
plot3 <- plot2 +
  scale_color_manual(
    values = c("Female" ="orange", "Male" = "red"))
```



ggplot2 Overview: facets

We split the plot into panels based on a categorical variable

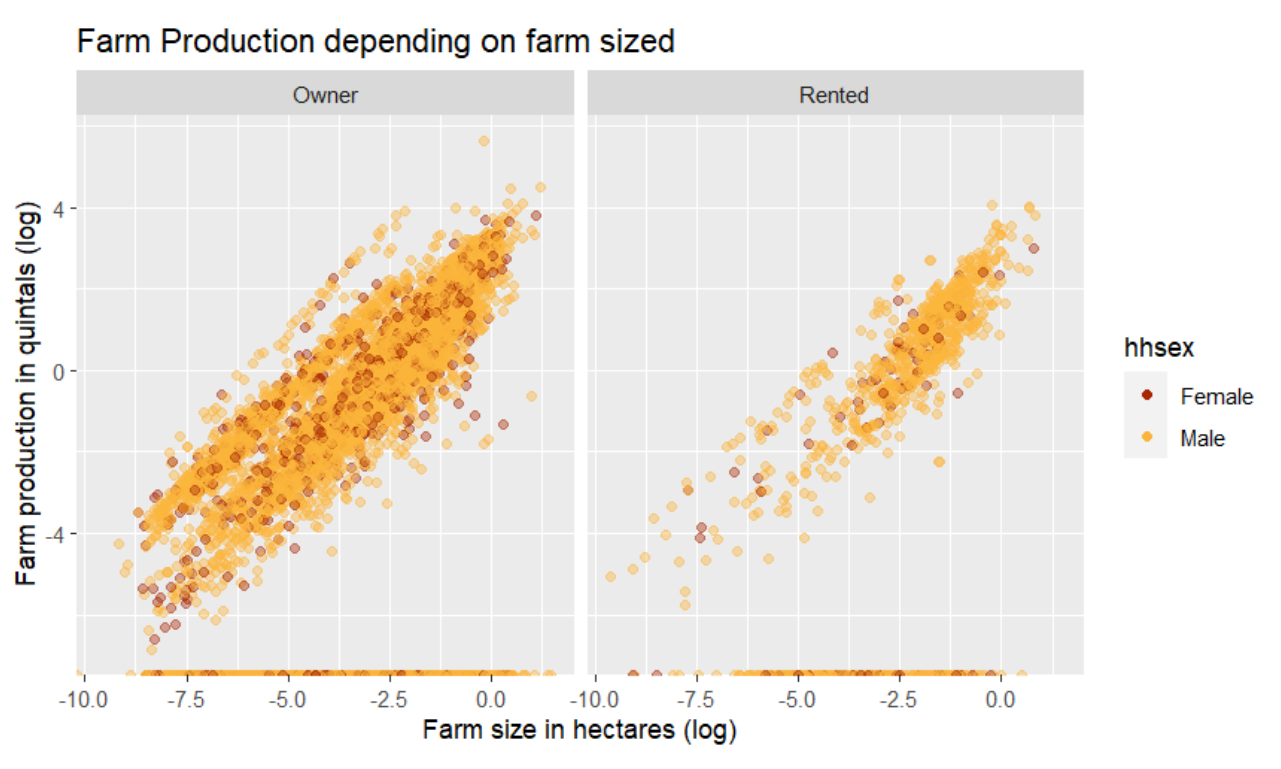
```
plot4 <- plot3 +
  facet_wrap(~OWNTYPE1,
    labeller = as_labeller(c("Rent" = "Rented", "Owner" = "Owner")))
```



ggplot2 Overview: labels

We change the labels

```
plot5 <- plot4 +
    labs(title = "Farm Production depending on farm size",
    x = "Farm size in hectares (log)",
    y = "Farm production in quintals (log)")
```

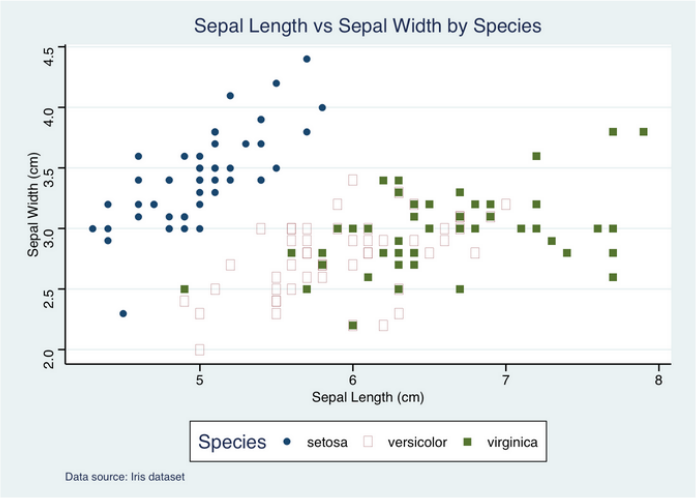


ggplot2 Overview: theme

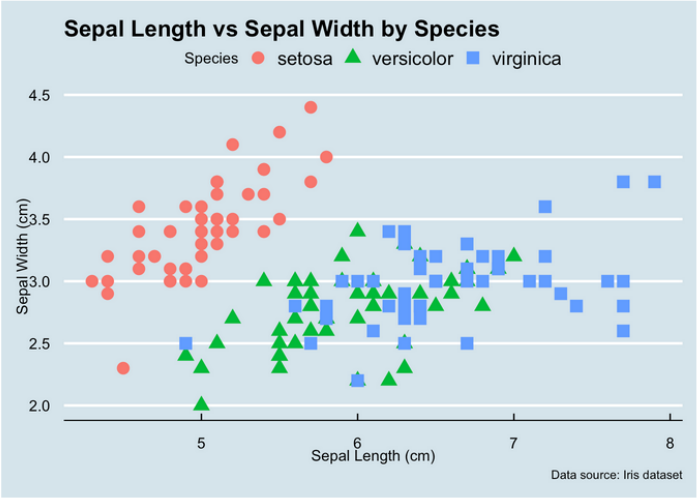
We can fine tune the graph look (see here for more details and examples), but the easiest past is to use themes from the package **ggthemes**.

A few examples here:

```
p + theme_stata() + scale_color_stata() + scale_shape_stata()
```




```
p + theme_economist()
```

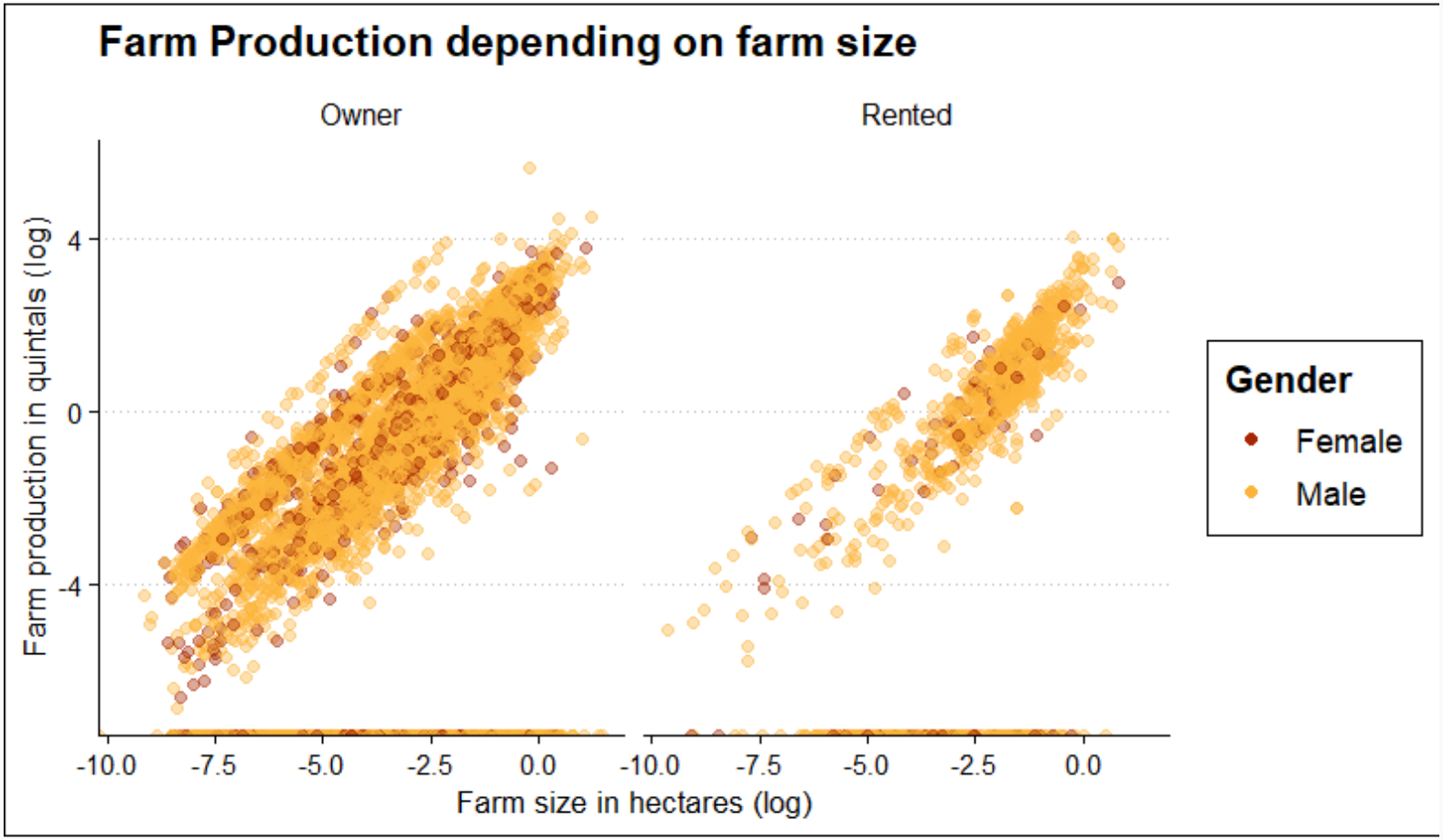


Have a look at this.

ggplot2 Overview: Final result

Final result with **theme_clean()**:

```
plot6 <- plot5 +
       theme_clean()
```



plot and ggplot2

- Base R include the **plot()** function that works, but becomes quite complicated once you want to do advanced or pretty graphs
- Hadley Wickham created **ggplot2** in 2005, a package to create graphics based the Grammar of Graphics

```
ggplot(data) +
    aes(aesthetics) +
    geometrics +
    ...
```

  - data: default dataset to use for plot
  - mapping: default list of aesthetic mappings to use for plot with the **aes()** function
  - geometrics: different functions **geom_*()** depending on how you want to visualize the data (**geom_point()**, **geom_col()**, **geom_line()**, ...)
  - other layers can be added with the '+' sign

See the cheat sheets for more details on ggplot2 potential

ggplot2: aesthetics

```
ggplot(data) +
    aes(x = ..., y = ..., color = ...) +
    geometrics +
    ...
```

- Link a variable and a dimension to represent it
  - **x** : variable for X-axis
  - **y** : variable for Y-axis
  - **color** : color (varying based on another variable)
  - **size** : size of points, thickness of lines
  - **alpha** : transparency
  - **linetype** : pattern of lines
  - **labels** : adding text on a plot or axes
  - **fill** : fill color
  - **shape** : shape of points
- *aesthetics* are passed down to the other functions, added with a '+' sign
- They become the default aesthetics for the latter functions
- You can define other *aesthetics* later in each **geom_*()** function

Exercise 2

- Charge the data "AGSS_15"
- Missing values for the quantity of fertilizer (D21B) are coded as 9999.999, recode them such that it does not alterate your analysis
- Generate 8 graphs
  1. Start with a basic scatter plot with the quantity of fertilizer (**D21B**) in x-axis, and total production (**PRODQ**) in y-axis using the **geom_point()** function for geometrics
  2. Replace y-axis with log of production
  3. Change color of points to red in the **geom_point()** options
  4. Change color depending on the household head sex **hhsex** (female/male) using the aesthetics
  5. Adjust **size** of points to 2 and reduce the transparency (**alpha**) to 0.1
  6. Change the **shape** of the points to square
  7. Store your graph in an object named 'basescatter'

ggplot2: geometrics

```
ggplot(data) +
    aes(x = ..., y = ...) +
    geom_density(aes(color = ...)) +
    ...
```

- Determine how the data will be plotted: see here for all the possibilities
- One continuous variable
  - **geom_density()**
  - **geom_histogram()**
  - **geom_area()**
  - **geom_dotplot()**
  - **geom_freqpoly()**
  - ...
- One discrete variable
  - **geom_bar()**
  - ...

ggplot2: geometrics

```
ggplot(data) +
    aes(x = ..., y = ...) +
    geom_point(aes(color = ...), size = 10) +
    ...
```

- One discrete and one continuous variable
  - **geom_col()**
  - **geom_boxplot()**
  - **geom_dotplot()**
  - **geom_violin()**
  - ...
- Two continuous variables
  - **geom_point()**
  - **geom_jitter()**
  - **geom_smooth()**
  - **geom_label()**
  - **geom_text()**
  - ...

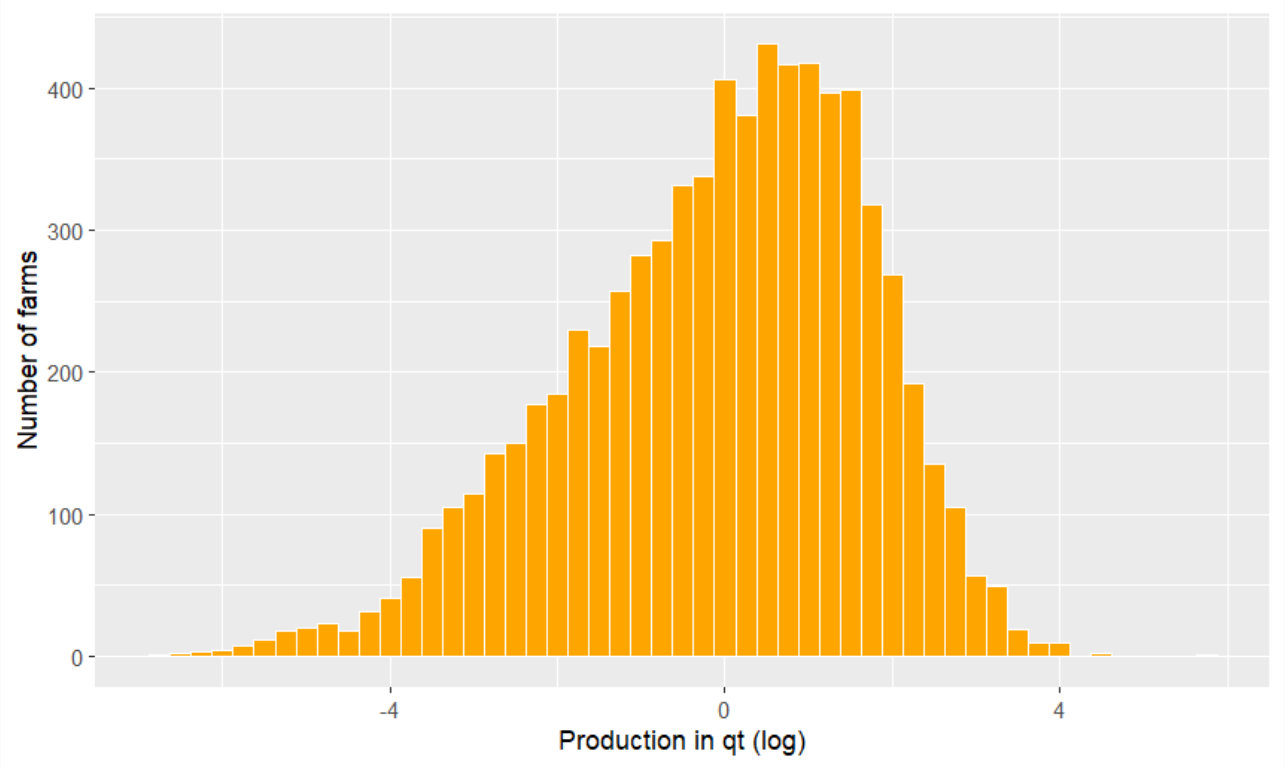
ggplot2: geometrics

```
ggplot(data) +
    aes(x = ..., y = ...) +
    stat_summary(fun = "median", geom = "point") +
    ...
```

- **stat_summary()** lets you summarize the data with a function (argument "fun") before plotting it with a geometric (argument "geom")
- Here you will plot the median with points

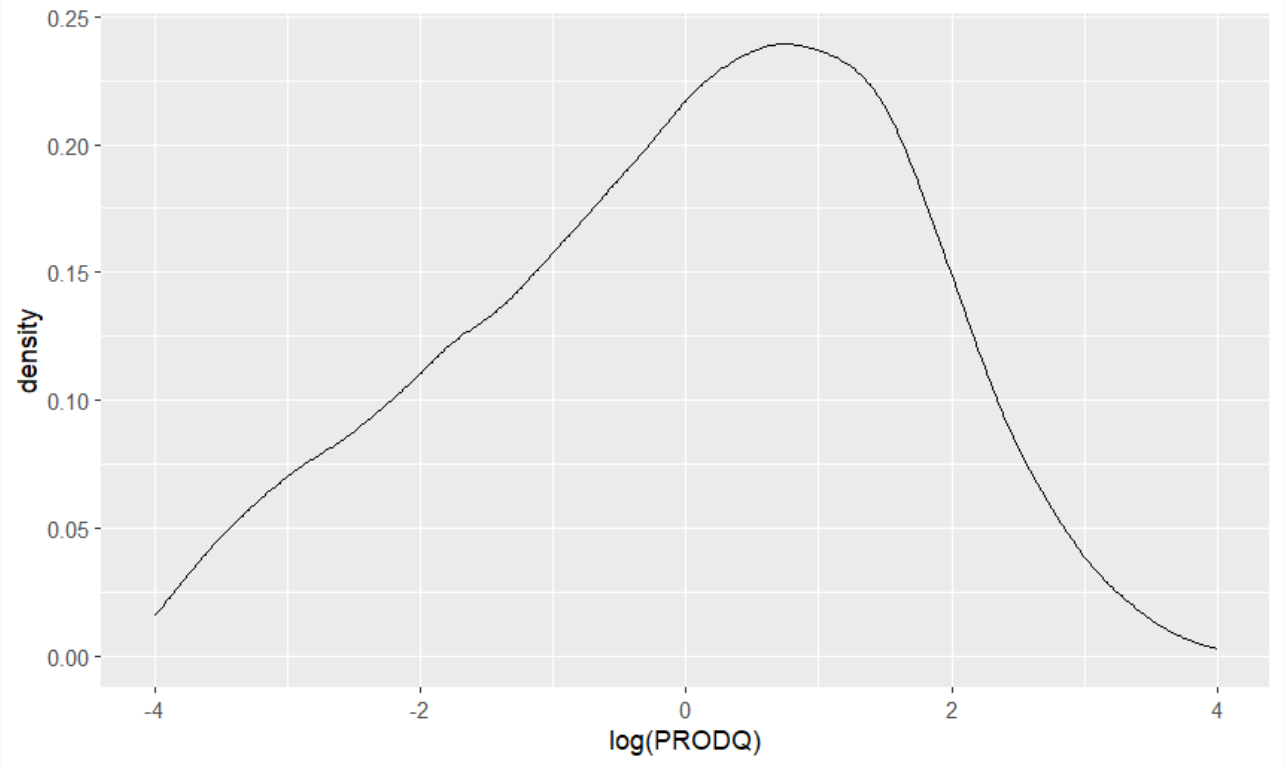
Plot one continuous variable: geom_hist()

```
ggplot(data) +
  aes(x = log(PRODQ) ) +
  geom_histogram()
```



Plot one continuous variable: geom_density()

```
ggplot(data, aes(x=log(PRODQ))) +
  geom_density() +
  xlim(-4, 4) # add limit
```



Exercise 3

Generate 4 plots

1. A density plot for farm size (**AREAH**)
   - Introduce limits between 0 and 1.2 using **xlim()**
   - Change the variable to log scale
2. A histogram for farm size (**AREAH**) with 50 bins
3. A bar graph for extension service access (**EXT1**)
4. A smoothed line plot with farm size (**AREAH**) in x-axis, and total production (**PRODQ**) in y-axis
   - Change the variable to log scale
5. Generate a bar plot with woreda identifier (**woreda_id**) on x-axis and total production (**PRODQ**) on y-axis
   - Adjust bar values so that it shows means, not sums
   - Add standard error of mean to the bars

ggplot2: geometrics position option

```
ggplot(data) +
    aes(x = ..., y = ...) +
    geom_*(position = "jitter") +
    ... +
    ...
```

- Position of geometrics
  - **identity** : default option, exactly where it is
  - **jitter**
  - **dodge**
  - **fill**
  - **stack**
  - **jitterdodge**
  - **nudge**

ggplot2: scale functions

- All aesthetics have scale functions:
  - scale_x_*() : modifying x-axis
  - scale_y_*() : modifying y-axis
  - scale_color_*()
  - scale_fill_*()
  - scale_size_*()
  - scale_shape_*()
  - scale_linetype_*()
- Where ***** corresponds to the type of data used
  - **scale_x_discrete()**
  - **scale_x_continuous()**
- labs() : if you want to change only axis labels

ggplot2: themes

- **theme_gray()** : the signature ggplot2 theme with a grey background and white gridlines
- **theme_bw()** : the classic dark-on-light ggplot2 theme
- **theme_linedraw()**: a theme with only black lines of various widths on white backgrounds
- **theme_light()** : a theme similar to **theme_linedraw()** but with light grey lines and axes
- **theme_dark()** : the dark cousin of **theme_light()**, with similar line sizes but a dark background
- **theme_minimal()** : a minimalistic theme with no background annotations
- **theme_classic()** : a classic-looking theme, with x and y-axis lines and no gridlines.
- **theme_void()** : a completely empty theme
- **theme()** : to modify all the parameters
- Further details in the ggplot2 website documentation

ggplot2: saving

- To save the current graph as a PNG

```
ggsave("graphname.png")
```

- To save the graph name "myscatter" as a PNG

```
ggsave("graphname.png", plot = "myscatter")
```

- **ggsave()** supports the following formats
  - eps
  - ps
  - tex (pictex)
  - pdf
  - jpeg
  - tiff
  - png
  - bmp
  - svg
  - wmf (windows only)

Exercise 4

Generate 6 graphs:

1. A scatter plot the quantity of fertilizer (**D21B**) in x-axis, and total production (**PRODQ**) in y-axis
2. Add labels to both axis using the scale functions **scale_x_*()** and **scale_y_*()**
3. Try to do the same using **labs()** function
4. Assign colors based on access to extension services (**EXT1**) and add labels to each of categories. Hint: use **scale_color_*()** function.
5. Save your plots in an object **scatplot**
6. Try different themes on saved plot **scatplot**

Sum up ggplot2

- Online documentation is good
- Get started : Summarizes how all the layers interact
- Articles : Aesthetic specifications
- Check list of a good graphic (article)
  - Title/Subtitle for the graph
  - Note and Source (Potential font issues when exporting)
  - Title for the axes (x might be obvious)
  - Comprehensive legend
  - Scale (Do you need logs ? Should you change the limits ?)
  - Background color, grids and ticks
  - Coherent pretty color palette across graphs (palettes online and accessibility)
- Use **theme_set()** to define a coherent design

$\Rightarrow$ Decrease the noise, focus on what you show

- List of references